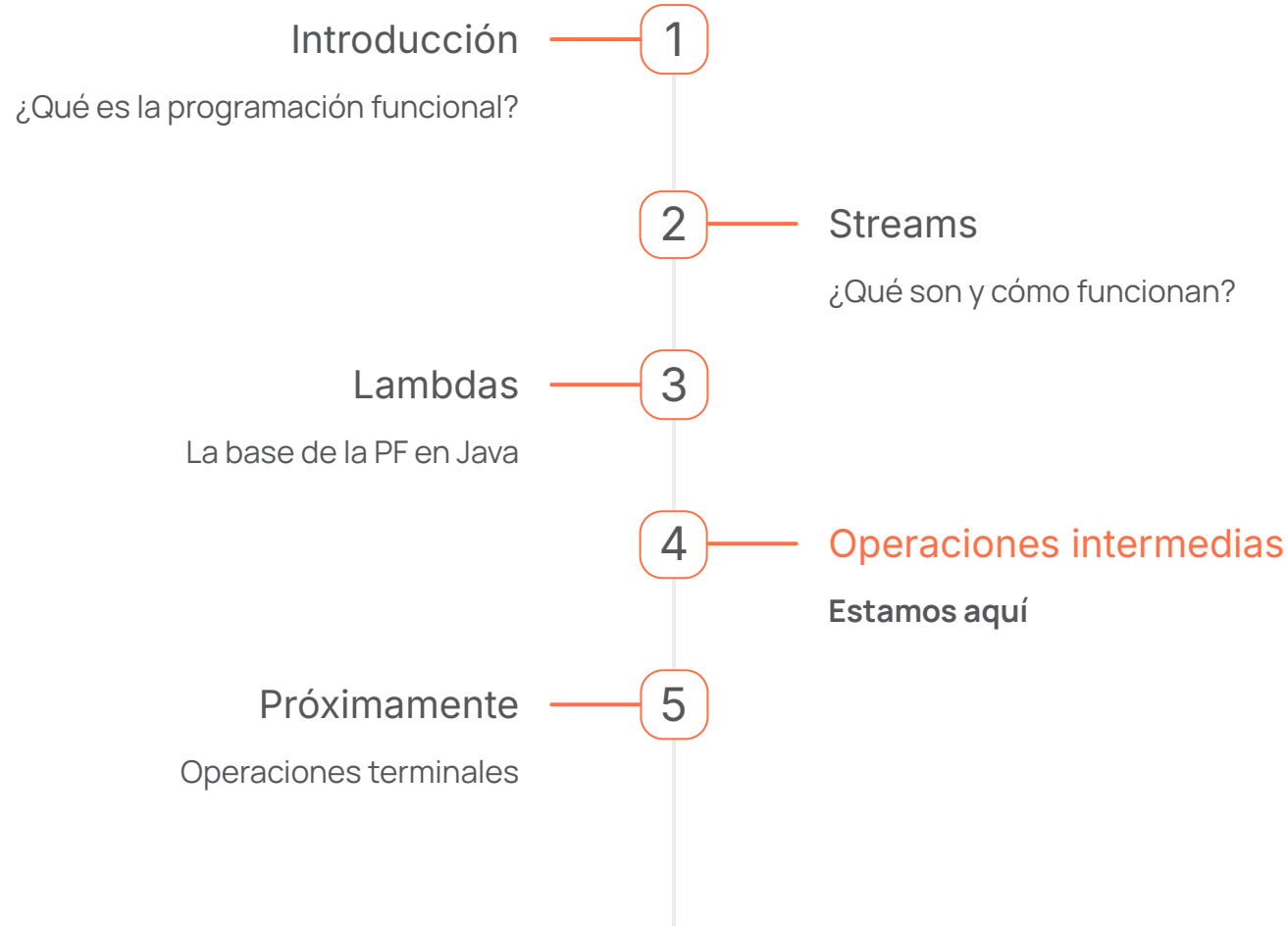




Operaciones Intermedias en Streams de Java

Transformar y filtrar datos de forma funcional

¿Dónde estamos en la serie?



Cuarto módulo de nuestra serie sobre Programación Funcional en Java

¿Qué son las operaciones intermedias?

Transformación

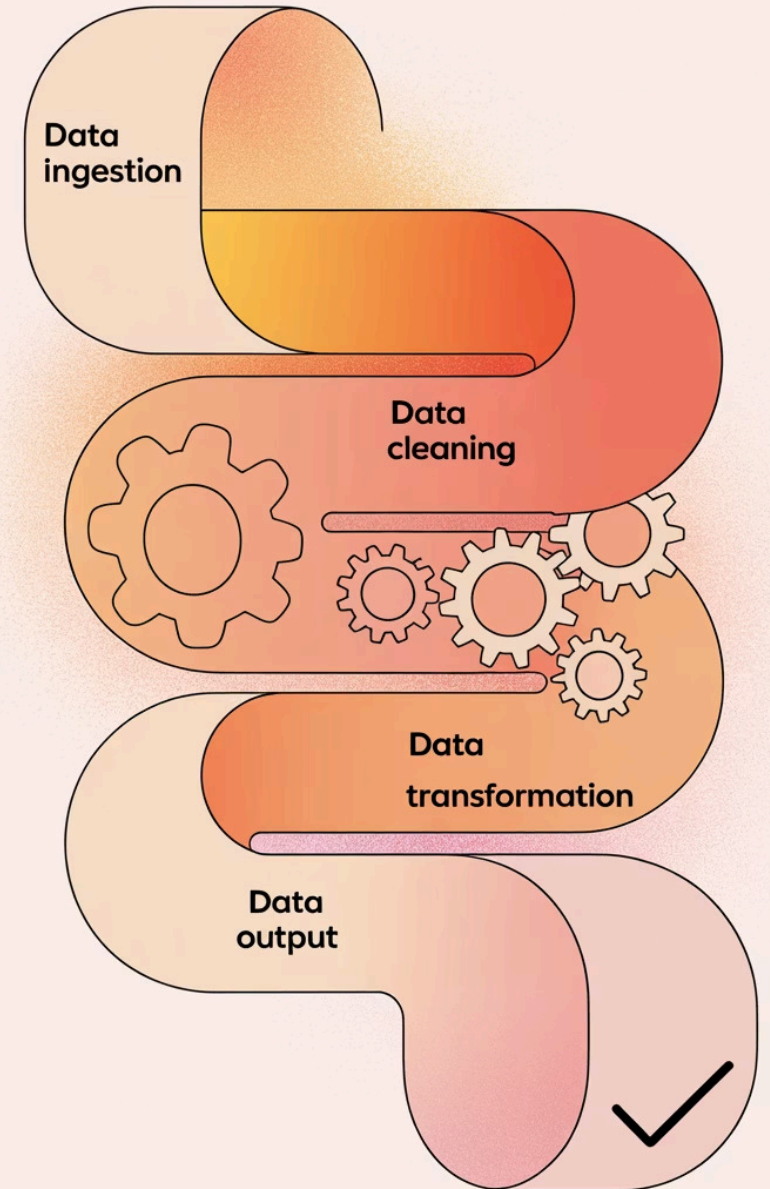
Transforman un Stream en otro Stream con datos modificados según nuestras necesidades

Preparación

Preparan los datos antes de obtener un resultado final mediante operaciones terminales

Evaluación perezosa (*lazy*)

Se ejecutan solo cuando se invoca una operación terminal, optimizando el rendimiento



Intermedias vs Terminales

Operaciones Intermedias

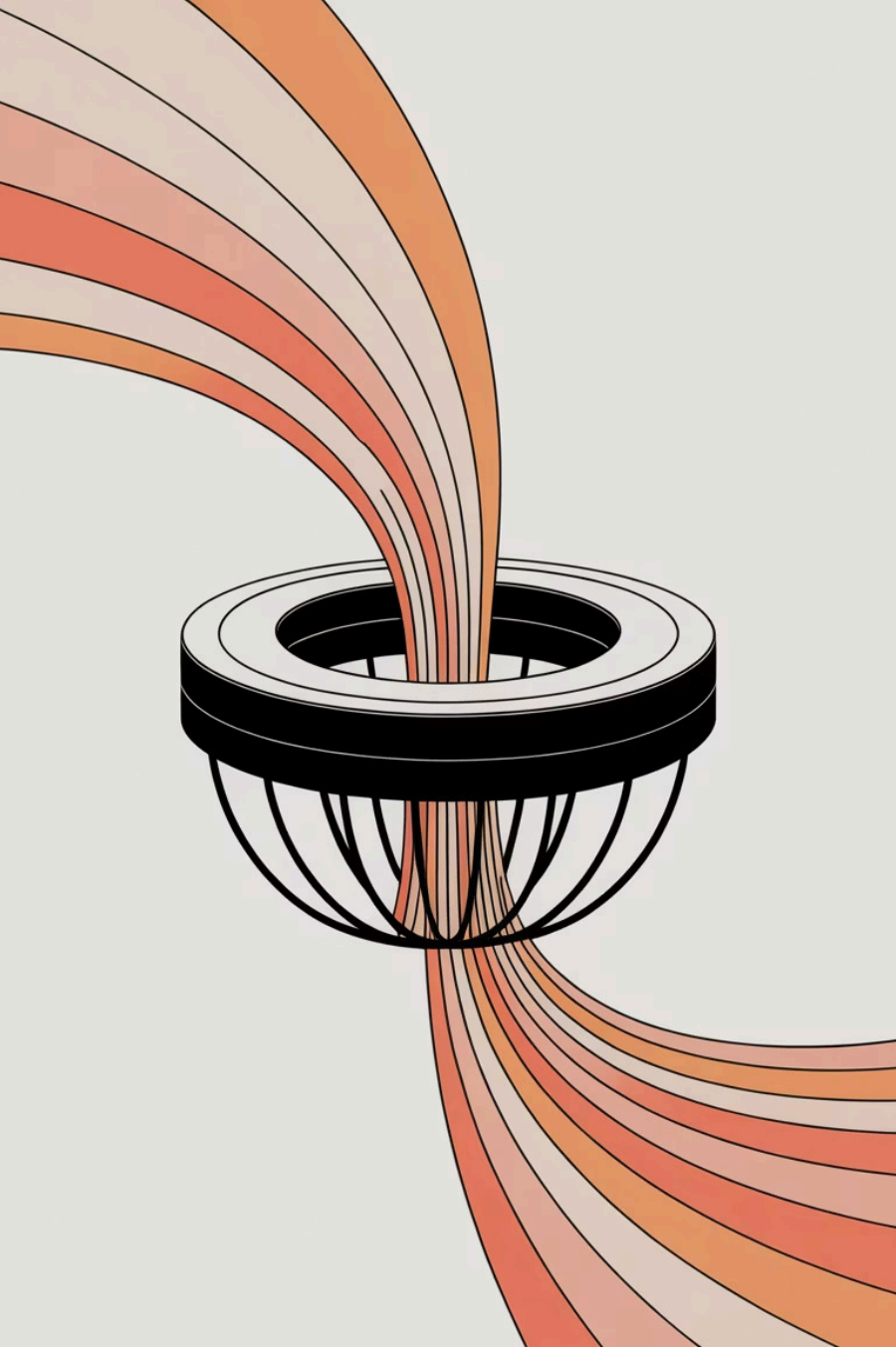
- Devuelven otro Stream
- Se pueden encadenar indefinidamente
- Evaluación perezosa (*lazy evaluation*)
- No producen resultados por sí solas

Ejemplos: `filter()`, `map()`, `distinct()`, `sorted()`

Operaciones Terminales

- Devuelven un resultado concreto
- Finalizan el procesamiento del Stream
- Activan la ejecución de las operaciones intermedias
- Solo puede haber una por Stream

Ejemplos: `forEach()`, `collect()`, `reduce()`, `count()`



filter(): Filtrando elementos

Permite seleccionar solo los elementos que cumplen con un predicado (función que devuelve boolean)

```
List numeros = List.of(1, 2, 3, 4, 5, 6, 7, 8, 9, 10);
```

```
List pares = numeros.stream()  
    .filter(n -> n % 2 == 0) // Solo números pares  
    .collect(Collectors.toList());
```

```
// Resultado: [2, 4, 6, 8, 10]
```

El predicado `n -> n % 2 == 0` evalúa si cada número es divisible por 2 (es par)

map(): Transformando elementos

Transforma cada elemento del Stream en otro elemento, posiblemente de un tipo diferente

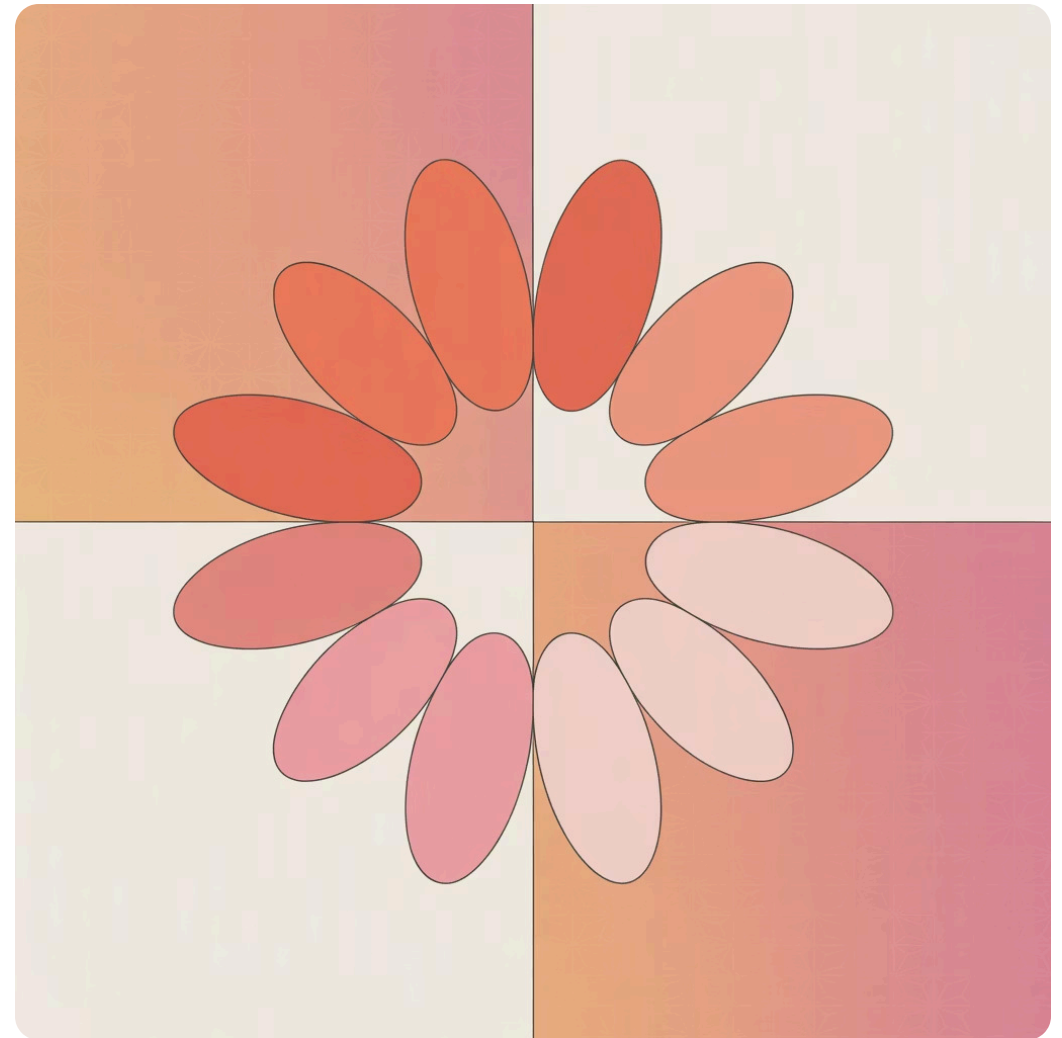
```
List nombres = List.of("ana", "juan", "elena");
```

```
List mayusculas = nombres.stream()  
    .map(String::toUpperCase)  
    .collect(Collectors.toList());
```

```
// Resultado: ["ANA", "JUAN", "ELENA"]
```

```
List longitudes = nombres.stream()  
    .map(String::length)  
    .collect(Collectors.toList());
```

```
// Resultado: [3, 4, 5]
```



❏ La función `map()` es una de las operaciones más poderosas y utilizadas en programación funcional. Permite transformar datos de forma clara y expresiva.

distinct(): Eliminando duplicados

Genera un nuevo Stream que contiene solo los elementos únicos, eliminando duplicados



Stream original

["manzana", "naranja", "manzana", "pera",
"naranja"]



distinct()

Filtra elementos duplicados utilizando el
método `equals()` para comparar



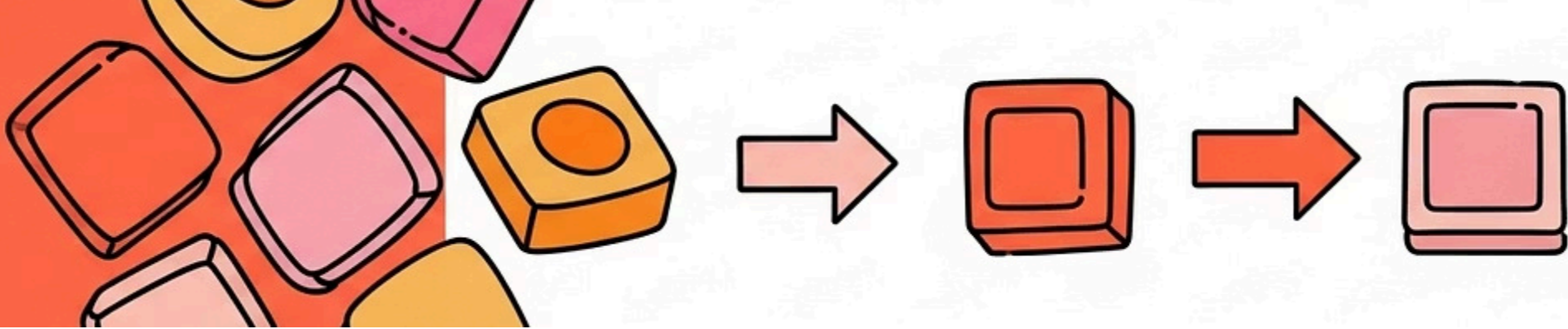
Stream resultante

["manzana", "naranja", "pera"]

```
List frutas = List.of("manzana", "naranja", "manzana", "pera", "naranja");
```

```
List frutasUnicas = frutas.stream()  
    .distinct()  
    .collect(Collectors.toList());
```

```
// Resultado: ["manzana", "naranja", "pera"]
```

sorted(): Ordenando elementos

Ordenamiento natural

Utiliza el orden natural de los elementos (implementan `Comparable`)

```
List numeros = List.of(5, 2, 8, 1, 3);  
  
List ordenados = numeros.stream()  
    .sorted()  
    .collect(Collectors.toList());  
  
// Resultado: [1, 2, 3, 5, 8]
```

Ordenamiento personalizado

Utiliza un `Comparator` personalizado

```
List palabras = List.of("Java", "es", "funcional");  
  
List porLongitud = palabras.stream()  
    .sorted(Comparator.comparing(String::length))  
    .collect(Collectors.toList());  
  
// Resultado: ["es", "Java", "funcional"]
```


Ejemplo Integrador: Frases a palabras únicas ordenadas

Problema: Dadas varias frases, extraer todas las palabras únicas y mostrarlas ordenadas alfabéticamente

1. Dividir frases en palabras con `flatMap`

Convertimos cada frase en un Stream de palabras y luego aplanamos todos en un solo Stream

2. Normalizar con `map`

Convertimos todas las palabras a minúsculas para comparar correctamente

3. Eliminar duplicados con `distinct`

Nos quedamos solo con palabras únicas

4. Ordenar con `sorted`

Ordenamos alfabéticamente

```
List frases = List.of(
    "Java es un lenguaje versátil",
    "La programación funcional en Java es poderosa",
    "Java tiene streams para procesamiento de datos"
);
```

```
List palabrasUnicas = frases.stream()
    .flatMap(frase -> Arrays.stream(frase.split(" ")))
    .map(String::toLowerCase)
    .distinct()
    .sorted()
    .collect(Collectors.toList());
```

Resumen

Operaciones Intermedias Principales

- `filter()`: Selecciona elementos según una condición
- `map()`: Transforma cada elemento
- `distinct()`: Elimina elementos duplicados
- `sorted()`: Ordena los elementos

Beneficios

- Encadenar operaciones aumenta la expresividad
- Código más declarativo y fácil de entender
- Evaluación perezosa optimiza el rendimiento

Próximo tema: Operaciones terminales (`forEach`, `collect`, `reduce...`)

